

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT: CYBER SECURITY

Aim: Implement algorithms to generate and verify message authentication code (MACs) for ensuring data integrity and authenticity.

Code: CLI

```
import hmac

import hashlib

import secrets

from cryptography.hazmat.primitives import cmac

from cryptography.hazmat.primitives.ciphers import algorithms

# ----- HMAC (hash-based MAC) -----

def generate_hmac(message: bytes):
    key = secrets.token_bytes(32)
    tag = hmac.new(key, message, hashlib.sha256).digest()
    print("Message:", message)
    print("Secret Key:", key.hex())
    print("HMAC:", tag.hex())
    return key, tag

def verify_hmac(key: bytes, message: bytes, tag: bytes) -> bool:
    valid = hmac.compare_digest(hmac.new(key, message, hashlib.sha256).digest(), tag)
    print("HMAC Valid:", valid)
    return valid

# ----- CMAC (AES block-cipher-based MAC) -----

def generate_cmac(message: bytes):
    key = secrets.token_bytes(16) # AES-128 key
```

Name: Dhairya Singh
Roll no: T119

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT: CYBER SECURITY

```
c = cmac.CMAC(algorithms.AES(key))
c.update(message)
tag = c.finalize()
print("Message:", message)
print("Secret Key:", key.hex())
print("CMAC:", tag.hex())
return key, tag

def verify_cmac(key: bytes, message: bytes, tag: bytes) -> bool:
    c = cmac.CMAC(algorithms.AES(key))
    c.update(message)
    try:
        c.verify(tag)
        print("CMAC Valid: True")
        return True
    except Exception:
        print("CMAC Valid: False")
        return False

# ----- Run: generate once, then verify whatever the user types -----

original_text = input("Enter message: ")
message = original_text.encode()

print("\n--- HMAC ---")
key1, tag1 = generate_hmac(message)

print("\n--- CMAC ---")
Name: Dhairya Singh
Roll no: T119
```

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT: CYBER SECURITY

```
key2, tag2 = generate_cmac(message)
```

```
# Verify step: re-read the message fresh, so verify checks REAL input,
```

```
# not the same variable used to generate (that was the bug before).
```

```
check_text = input("\nEnter message to verify: ")
```

```
check_message = check_text.encode()
```

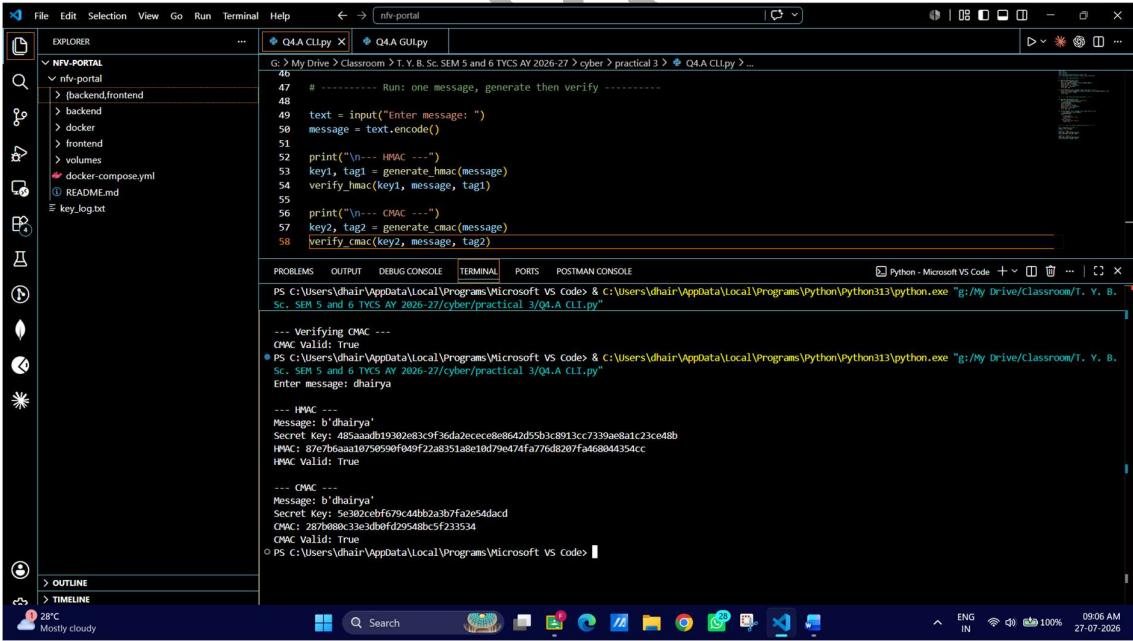
```
print("\n--- Verifying HMAC ---")
```

```
verify_hmac(key1, check_message, tag1)
```

```
print("\n--- Verifying CMAC ---")
```

```
verify_cmac(key2, check_message, tag2)
```

Output:



```
G:\> My Drive > Classroom > T. Y. B. Sc. SEM 5 and 6 TYCS AY 2026-27 > cyber > practical 3 > Q4A CLI.py > ...
46
47 # ----- Run: one message, generate then verify -----
48
49 text = input("Enter message: ")
50 message = text.encode()
51
52 print("\n--- HMAC ---")
53 key1, tag1 = generate_hmac(message)
54 verify_hmac(key1, message, tag1)
55
56 print("\n--- CMAC ---")
57 key2, tag2 = generate_cmac(message)
58 verify_cmac(key2, message, tag2)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
Python - Microsoft VS Code
PS C:\Users\dhairya\AppData\Local\Programs\Microsoft VS Code> & C:\Users\dhairya\AppData\Local\Programs\Python\Python313\python.exe "g:/My Drive/Classroom/T. Y. B. Sc. SEM 5 and 6 TYCS AY 2026-27/cyber/practical 3/Q4A CLI.py"

--- Verifying CMAC ---
CMAC Valid: True
PS C:\Users\dhairya\AppData\Local\Programs\Microsoft VS Code> & C:\Users\dhairya\AppData\Local\Programs\Python\Python313\python.exe "g:/My Drive/Classroom/T. Y. B. Sc. SEM 5 and 6 TYCS AY 2026-27/cyber/practical 3/Q4A CLI.py"
Enter message: dhairya

--- HMAC ---
Message: b'dhairya'
Secret Key: 485aaad19302e83c9f36da2ecee8e8642f59b3c8913cc7330ae8a1c23c048b
HMAC: 8f9796aa10759590f049f22a8351a8e18d79e474fa776d8207fa468044354cc
HMAC Valid: True

--- CMAC ---
Message: b'dhairya'
Secret Key: 5e302ebf79c44b2a2b7fa2554dad
CMAC: 28f0880c33e3db0fd29548bc5f233534
CMAC Valid: True
PS C:\Users\dhairya\AppData\Local\Programs\Microsoft VS Code>
```

Name: Dhairya Singh
Roll no: T119

GUI: INPUT

```
import tkinter as tk

from tkinter import messagebox

import hmac

import hashlib

import secrets

from cryptography.hazmat.primitives import cmac

from cryptography.hazmat.primitives.ciphers import algorithms

# ----- HMAC (hash-based MAC) -----

def generate_hmac(message: bytes):

    key = secrets.token_bytes(32)

    tag = hmac.new(key, message, hashlib.sha256).digest()

    return key, tag

def verify_hmac(key: bytes, message: bytes, tag: bytes) -> bool:

    return hmac.compare_digest(hmac.new(key, message, hashlib.sha256).digest(), tag)

# ----- CMAC (AES block-cipher-based MAC) -----

def generate_cmac(message: bytes):

    key = secrets.token_bytes(16) # AES-128 key

    c = cmac.CMAC(algorithms.AES(key))

    c.update(message)

    tag = c.finalize()

    return key, tag
```

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT: CYBER SECURITY

```
def verify_cmac(key: bytes, message: bytes, tag: bytes) -> bool:

    c = cmac.CMAC(algorithms.AES(key))

    c.update(message)

    try:

        c.verify(tag)

        return True

    except Exception:

        return False


# ----- shared state (holds last generated keys/tags) -----

state = {"message": None, "hmac_key": None, "hmac_tag": None,
        "cmac_key": None, "cmac_tag": None}


# ----- GUI actions -----

def do_generate():

    text = entry.get().strip()

    if not text:

        messagebox.showwarning("Input needed", "Please enter a message.")

        return

    message = text.encode()

    hkey, htag = generate_hmac(message)

    ckey, ctag = generate_cmac(message)

    state["message"] = message

    state["hmac_key"], state["hmac_tag"] = hkey, htag
```

Name: Dhairya Singh
Roll no: T119

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT: CYBER SECURITY

```
state["cmac_key"], state["cmac_tag"] = ckey, ctag
```

```
output.delete("1.0", tk.END)
```

```
output.insert(tk.END, f"Message: {text}\n")
```

```
output.insert(tk.END, f"Encoded: {message}\n\n")
```

```
output.insert(tk.END, "--- HMAC Generated ---\n")
```

```
output.insert(tk.END, f"Secret Key: {hkey.hex()}\n")
```

```
output.insert(tk.END, f"HMAC: {htag.hex()}\n\n")
```

```
output.insert(tk.END, "--- CMAC Generated ---\n")
```

```
output.insert(tk.END, f"Secret Key: {ckey.hex()}\n")
```

```
output.insert(tk.END, f"CMAC: {ctag.hex()}\n")
```

```
def do_verify():
```

```
    if state["hmac_key"] is None:
```

```
        messagebox.showwarning("Nothing to verify", "Generate the MACs first.")
```

```
        return
```

```
text = entry.get().strip()
```

```
if not text:
```

```
    messagebox.showwarning("Input needed", "Please enter a message to verify.")
```

```
    return
```

```
message = text.encode() # re-read current box content, not the stored one
```

```
hmac_valid = verify_hmac(state["hmac_key"], message, state["hmac_tag"])
```

```
cmac_valid = verify_cmac(state["cmac_key"], message, state["cmac_tag"])
```

```
output.insert(tk.END, f"HMAC Valid: {hmac_valid}\n")
```

Name: Dhairya Singh

Roll no: T119

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT: CYBER SECURITY

```
output.insert(tk.END, f"CMAC Valid: {cmac_valid}\n")
```

```
# ----- GUI layout -----
```

```
root = tk.Tk()
```

```
root.title("MAC Generator - HMAC & CMAC")
```

```
root.geometry("560x480")
```

```
tk.Label(root, text="Enter Message:").pack(pady=(10, 0))
```

```
entry = tk.Entry(root, width=60)
```

```
entry.pack(pady=5)
```

```
tk.Button(root, text="Generate HMAC & CMAC", command=do_generate).pack(pady=5)
```

```
tk.Button(root, text="Verify HMAC & CMAC", command=do_verify).pack(pady=5)
```

```
output = tk.Text(root, width=68, height=18, wrap="word")
```

```
output.pack(pady=10)
```

```
root.mainloop()
```

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT: CYBER SECURITY

Output:

The screenshot shows a Python GUI application titled "MAC Generator - HMAC & CMAC" running in a Windows environment. The application has a text input field containing "dhairya", a "Generate HMAC & CMAC" button, and a "Verify HMAC & CMAC" button. The output area displays the results of the operations:

```
Message: dhairya
Encoded: b'dhairya'

--- HMAC Generated ---
Secret Key: e30c459f06298758672d66824fe922482566bfa7cb04d50e5c45d98f6bb41ae
HMAC: c3a92c325e9ad8bado3d03d9e281ff926e47542803114ca9e220b694c9750872

--- CMAC Generated ---
Secret Key: c551850644908e7643f68584e842244
CMAC: feb2dad6993c02648d4c43b7d3c0b5e0
HMAC Valid: True
CMAC Valid: True

--- Verify ---
Message: b'dhairya'
Secret Key: 485aaad019302683c5
HMAC: 87e7b6aa10750590f049f22
HMAC Valid: True

--- CMAC ---
Message: b'dhairya'
Secret Key: 5e302ceb679c4bb2a3b7fa2e54dad
CMAC: 2876808a3e3db0fd2954abc5f233534
CMAC Valid: True
```

The background shows a VS Code editor with a Python script for the application and a terminal window.

Name: Dhairya Singh
Roll no: T119